

preprocessing_life_style_data_v1.2

November 11, 2025

1 Preprocessing - life_style_data.duckdb

1.1 Importation des librairies essentielles

```
[61]: import duckdb
import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler
```

1.2 Nettoyage des données

1.2.1 Contexte & objectifs

Ce chapitre prépare le dataset life_style_data pour la modélisation. Objectifs : - Corriger les types (notamment les décimales avec virgules) ; - Quantifier et traiter les valeurs manquantes ; - Détecter/supprimer les doublons ; - Poser des règles de plausibilité métier (sans censurer) ; - Produire un rapport de nettoyage et des artefacts (dataset nettoyé).

Entrée : table DuckDB life_style_data (chargée via DBBeaver). Sorties : df_clean (Parquet/CSV), tableaux de suivi (manquants, conversions, doublons, flags).

1.2.2 Imports, paramètres, chargement depuis DuckDB

```
[62]: # Définition des chemins (adaptés à ton arborescence GitHub)
# Notebook : C:\Users\fbac\Desktop\Projets\Dev\GitHub\train.
#         ↪ me\src\notebooks\preprocessing\life_style_data
# Base      : C:\Users\fbac\Desktop\Projets\Dev\GitHub\train.
#         ↪ me\src\data\raw\life_style_data

# Le notebook s'exécute depuis son répertoire → on peut repartir du cwd
current_dir = Path.cwd()
project_root = current_dir.parents[3] # remonte jusqu'à "train.me"

# Chemins complets
db_path = project_root / "src" / "data" / "raw" / "life_style_data" / "
#         ↪ "life_style_data"
```

```

export_dir = project_root / "src" / "data" / "processed" / "life_style_data"

# Connexion à la base DuckDB
con = duckdb.connect(str(db_path))

# Chargement de la table
df = con.execute("SELECT * FROM life_style_data").fetchdf()

# Vérification du chargement
shape_before = df.shape
print(f" Données chargées : {shape_before[0]} lignes, {shape_before[1]} ↵
    ↵ colonnes")
display(df.head(3))

```

Données chargées : 20000 lignes, 54 colonnes

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	\
0	34.91	Male	65.27	1.62	188.58	157.65	69.05	
1	23.37	Female	56.41	1.55	179.43	131.75	73.18	
2	33.20	Female	58.98	1.67	175.04	123.95	54.96	

	Session_Duration (hours)	Calories_Burned	Workout_Type	...	\
0	1.00	1080.90	Strength	...	
1	1.37	1809.91	HIIT	...	
2	0.91	802.26	Cardio	...	

	cal_from_macros	pct_carbs	protein_per_kg	pct_HRR	pct_maxHR	\
0	2139.59	0.500432	1.624789	0.741237	0.835985	
1	1711.65	0.500850	1.514093	0.551247	0.734270	
2	1965.92	0.500610	1.663445	0.574534	0.708124	

	cal_balance	lean_mass_kg	expected_burn	Burns Calories (per 30 min)_bc	\
0	725.10	47.777394	685.1600	7.260425e+19	
1	-232.91	40.809803	978.6184	1.020506e+20	
2	805.74	44.635580	654.5266	1.079607e+20	

	Burns_Calories_Bin
0	Medium
1	High
2	High

[3 rows x 54 columns]

1.2.3 Audit qualité initial

On dresse un état des lieux : - Répartition des types et cardinalités ; - Valeurs manquantes (%) par colonne ; - Doublons potentiels.

Cet instantané nous sert de référence avant toute modification.

```
[63]: # 1.2 Audit qualité initial
missing_pct = df.isna().mean().sort_values(ascending=False) * 100
dup_count = df.duplicated().sum()

summary_types = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "n_unique": df.nunique(),
    "missing_%": df.isna().mean() * 100
}).sort_values("missing_", ascending=False)

print(f"• Doublons détectés (avant): {dup_count}")
display(missing_pct.to_frame("missing_").head(20))
display(summary_types.head(20))
```

• Doublons détectés (avant): 0

	missing_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0
Target Muscle Group	0.0
Equipment Needed	0.0
Difficulty Level	0.0
Type of Muscle	0.0
Gender	0.0
Workout	0.0
BMI_calc	0.0
cal_from_macros	0.0
pct_carbs	0.0
protein_per_kg	0.0

	dtype	n_unique	missing_%
Age	float64	3950	0.0
Body Part	object	7	0.0
prep_time_min	float64	5384	0.0
cook_time_min	float64	9406	0.0
rating	float64	414	0.0
Name of Exercise	object	55	0.0
Sets	float64	30	0.0
Reps	float64	654	0.0
Benefit	object	49	0.0
Burns Calories (per 30 min)	float64	5766	0.0

Target Muscle Group	object	36	0.0
Equipment Needed	object	20	0.0
Difficulty Level	object	3	0.0
Type of Muscle	object	13	0.0
Gender	object	2	0.0
Workout	object	53	0.0
BMI_calc	float64	18158	0.0
cal_from_macros	float64	18808	0.0
pct_carbs	float64	19982	0.0
protein_per_kg	float64	19986	0.0

1.2.4 Correction de types (décimales à virgule, espaces insécables)

Dans de nombreux exports, les colonnes numériques arrivent en texte (object) avec : - des virgules à la place du point décimal ; - des espaces insécables (00A0) ou espaces.

On corrige proprement pour garantir la cohérence numérique.

```
[64]: # 1.3 Conversion robuste des colonnes numériques candidates (si présentes)
num_candidates = [
    "Age", "Weight (kg)", "Height (m)", "Max_BPM", "Avg_BPM", "Resting_BPM",
    "Session_Duration (hours)", "Fat_Percentage", "Water_Intake (liters)",
    "Workout_Frequency (days/week)", "BMI", "Carbs", "Proteins", "Fats", "Calories",
    ↵
    ↵ "sugar_g", "sodium_mg", "cholesterol_mg", "serving_size_g", "prep_time_min", "cook_time_min",
    ↵
    ↵ "rating", "BMI_calc", "cal_from_macros", "pct_carbs", "protein_per_kg", "pct_HRR", "pct_maxHR",
    "cal_balance", "lean_mass_kg", "expected_burn", "Burns Calories (per 30
    ↵ min)", "Calories_Burned"
]

convert_log = []
df_conv = df.copy()

for c in [col for col in num_candidates if col in df_conv.columns]:
    if df_conv[c].dtype == "object":
        before_na = df_conv[c].isna().sum()
        df_conv[c] = (df_conv[c]
            .astype(str)
            .str.replace(",", ".", regex=False) # virgule point
            .str.replace("\u00A0", "", regex=False) # espace insécable
            .str.strip()
        )
        df_conv[c] = pd.to_numeric(df_conv[c], errors="coerce")
        after_na = df_conv[c].isna().sum()
        convert_log.append({"column": c, "was_object": True, "na_before": ↵
        ↵ before_na, "na_after": after_na})
```

```
convert_report = pd.DataFrame(convert_log).sort_values("column") if convert_log
↳ else pd.DataFrame(columns=["column", "was_object", "na_before", "na_after"])
display(convert_report)
```

Empty DataFrame

Columns: [column, was_object, na_before, na_after]

Index: []

1.2.5 Règles de plausibilité (flags non bloquants)

On applique des bornes métier pour repérer des valeurs possiblement aberrantes. On ne supprime pas : on flague pour informer la modélisation et documenter les décisions.

Exemples (paramétrables) : - Age [10, 90] - Max_BPM [80, 230] - Session_Duration (hours) [0.1, 6] - BMI [10, 60]

```
[65]: # 1.4 Flags de plausibilité (ne supprime rien)
df_flag = df_conv.copy()

def safe_between(s, low, high):
    return s.between(low, high) if s.notna().any() else pd.Series(False,
↳ index=s.index)

flags = {}
if "Age" in df_flag:
    flags["Age_out"] =
↳ safe_between(df_flag["Age"], 10, 90)
if "Max_BPM" in df_flag:
    flags["BPM_out"] =
↳ safe_between(df_flag["Max_BPM"], 80, 230)
if "Session_Duration (hours)" in df_flag: flags["SessDur_out"] =
↳ safe_between(df_flag["Session_Duration (hours)"], 0.1, 6)
if "BMI" in df_flag:
    flags["BMI_out"] =
↳ safe_between(df_flag["BMI"], 10, 60)

flag_counts = {k: v.fillna(False).sum() for k, v in flags.items()}
flag_table = pd.DataFrame.from_dict(flag_counts, orient="index",
↳ columns=["nb_outliers"]).sort_index()
display(flag_table)
```

	nb_outliers
Age_out	0
BMI_out	0
BPM_out	0
SessDur_out	0

1.2.6 Imputation des valeurs manquantes (baseline)

Stratégie simple et robuste (baseline reproductible) : - Numériques → médiane (moins sensible aux outliers) ; - Catégorielles → mode (valeur la plus fréquente, sinon “Unknown”).

Cette baseline est suffisante pour itérer ; on pourra spécialiser par la suite.

```
[66]: # 1.5 Imputation des manquants
df_imp = df_flag.copy()

num_cols = df_imp.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = [c for c in df_imp.columns if c not in num_cols]

# Numériques → médiane
for c in num_cols:
    if df_imp[c].isna().any():
        df_imp[c] = df_imp[c].fillna(df_imp[c].median())

# Catégorielles → mode (ou "Unknown")
for c in cat_cols:
    if df_imp[c].isna().any():
        m = df_imp[c].mode(dropna=True)
        df_imp[c] = df_imp[c].fillna(m.iloc[0] if not m.empty else "Unknown")

missing_before = missing_pct # gardé depuis 1.2
missing_after = df_imp.isna().mean().sort_values(ascending=False) * 100

print(" Imputation effectuée.")
display(pd.DataFrame({"missing_before_%": missing_before}).head(10))
display(pd.DataFrame({"missing_after_%": missing_after}).head(10))
```

Imputation effectuée.

	missing_before_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0

	missing_after_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0

Benefit	0.0
Burns Calories (per 30 min)	0.0

1.2.7 Doublons & incohérences simples

On élimine les doublons stricts (lignes identiques).

Cette opération stabilise la taille du corpus et évite des biais à l'entraînement.

```
[67]: # 1.6 Doublons
rows_before = df_imp.shape[0]
dup_total = df_imp.duplicated().sum()
df_clean = df_imp.drop_duplicates().reset_index(drop=True)
rows_after = df_clean.shape[0]

clean_report = pd.DataFrame({
    "rows_before": [rows_before],
    "rows_after": [rows_after],
    "duplicates_removed": [dup_total],
    "cols": [df_clean.shape[1]]
})
display(clean_report)
```

	rows_before	rows_after	duplicates_removed	cols
0	20000	20000	0	54

1.2.8 Rapport de nettoyage (synthèse consultant)

Nous consolidons les éléments clés pour traçabilité et communication : - Colonnes converties object → numeric ; - Avant/Après des valeurs manquantes ; - Nombre de doublons supprimés ; - Flags de plausibilité relevés.

Ce rapport sert de référence pour justifier les choix et préparer la suite (encodage, scaling).

```
[68]: # 1.7 Rapport consolidé
sections = {
    "convert_report": convert_report,
    "missing_before_% (top 20)": missing_before.to_frame("missing_").head(20),
    "missing_after_% (top 20)": missing_after.to_frame("missing_").head(20),
    "plausibility_flags": flag_table,
    "clean_report": clean_report
}

for title, df_sec in sections.items():
    print("\n" + "="*30 + f" {title} " + "="*30)
    display(df_sec)
```

```
===== convert_report =====
```

```
Empty DataFrame
Columns: [column, was_object, na_before, na_after]
Index: []
```

```
===== missing_before_% (top 20)
=====
```

	missing_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0
Target Muscle Group	0.0
Equipment Needed	0.0
Difficulty Level	0.0
Type of Muscle	0.0
Gender	0.0
Workout	0.0
BMI_calc	0.0
cal_from_macros	0.0
pct_carbs	0.0
protein_per_kg	0.0

```
===== missing_after_% (top 20)
=====
```

	missing_%
Age	0.0
Body Part	0.0
prep_time_min	0.0
cook_time_min	0.0
rating	0.0
Name of Exercise	0.0
Sets	0.0
Reps	0.0
Benefit	0.0
Burns Calories (per 30 min)	0.0
Target Muscle Group	0.0
Equipment Needed	0.0
Difficulty Level	0.0
Type of Muscle	0.0
Gender	0.0

Workout	0.0
BMI_calc	0.0
cal_from_macros	0.0
pct_carbs	0.0
protein_per_kg	0.0

===== plausibility_flags =====

	nb_outliers
Age_out	0
BMI_out	0
BPM_out	0
SessDur_out	0

===== clean_report =====

	rows_before	rows_after	duplicates_removed	cols
0	20000	20000	0	54

1.2.9 Export des artefacts (dataset nettoyé)

On persiste le dataset nettoyé pour les étapes suivantes (encodage, scaling, split) et pour reproductibilité : - life_style_data_clean.parquet (format recommandé) ; - Optionnel : life_style_data_clean.csv.

```
[69]: # 1.8 Export artefacts
parquet_path = export_dir / "life_style_data_clean.parquet"
csv_path      = export_dir / "life_style_data_clean.csv"

df_clean.to_parquet(parquet_path, index=False, engine="fastparquet")
df_clean.to_csv(csv_path, index=False)

print(f" Export parquet : {parquet_path}")
print(f" Export csv      : {csv_path}")
print(f" Shape final     : {df_clean.shape}")
```

Export parquet : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_clean.parquet

Export csv : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_clean.csv

Shape final : (20000, 54)

1.2.10 Critères d'acceptation & décisions

- Types numériques/catégoriels conformes au dictionnaire ;
- Manquants résiduels documentés et seuils tolérés (par défaut, 5%) ;
- Doublons supprimés (ou chiffrés et justifiés si conservés) ;
- Flags de plausibilité suivis (pas de censure sans décision métier explicite).

Décisions à garder au journal : - Colonnes converties, colonnes fortement manquantes et stratégie appliquée ; - Seuils de plausibilité retenus et impact (aucune ligne supprimée à ce stade) ; - Lieux d'export des artefacts.

1.3 Encodage des variables catégorielles

1.3.1 Objectifs

Cette étape consiste à transformer les variables catégorielles du jeu de données nettoyé en représentations numériques exploitables par les algorithmes de Machine Learning. L'encodage rend les colonnes qualitatives compatibles avec les modèles tout en préservant l'information sémantique. Deux approches principales sont appliquées selon la nature des catégories : - Encodage ordinal pour les variables hiérarchiques (ex. niveau d'expérience) ; - Encodage one-hot pour les variables nominales (ex. type d'exercice, partie du corps, équipement).

1.3.2 Chargement du dataset nettoyé

```
[70]: # Charger les artefacts nettoyés
df_clean = pd.read_parquet(parquet_path)

print(f" Dataset chargé : {df_clean.shape[0]} lignes, {df_clean.shape[1]} ↵
      ↵ colonnes")
df_clean.head(3)
```

Dataset chargé : 20000 lignes, 54 colonnes

```
[70]:      Age  Gender  Weight (kg)  Height (m)  Max_BPM  Avg_BPM  Resting_BPM  \
0   34.91   Male     65.27      1.62    188.58   157.65      69.05
1   23.37  Female     56.41      1.55    179.43   131.75      73.18
2   33.20  Female     58.98      1.67    175.04   123.95      54.96

      Session_Duration (hours)  Calories_Burned  Workout_Type  ...  \
0                1.00          1080.90      Strength  ...
1                1.37          1809.91           HIIT  ...
2                0.91           802.26          Cardio  ...

      cal_from_macros  pct_carbs  protein_per_kg  pct_HRR  pct_maxHR  \
0          2139.59    0.500432    1.624789    0.741237    0.835985
1          1711.65    0.500850    1.514093    0.551247    0.734270
2          1965.92    0.500610    1.663445    0.574534    0.708124

      cal_balance  lean_mass_kg  expected_burn  Burns Calories (per 30 min)_bc  \
0          725.10    47.777394    685.1600          7.260425e+19
1         -232.91    40.809803    978.6184          1.020506e+20
2          805.74    44.635580    654.5266          1.079607e+20

      Burns_Calories_Bin
0                Medium
1                High
```

[3 rows x 54 columns]

1.3.3 Identification des variables catégorielles

On repère les variables non numériques à encoder. L'objectif est de distinguer : - les variables nominales (non ordonnées), - les variables ordinales (possédant une hiérarchie logique).

```
[71]: # Identification des colonnes catégorielles
cat_cols = df_clean.select_dtypes(exclude=["number"]).columns.tolist()
print(f" {len(cat_cols)} variables catégorielles détectées :")
cat_cols
```

15 variables catégorielles détectées :

```
[71]: ['Gender',
      'Workout_Type',
      'meal_name',
      'meal_type',
      'diet_type',
      'cooking_method',
      'Name of Exercise',
      'Benefit',
      'Target Muscle Group',
      'Equipment Needed',
      'Difficulty Level',
      'Body Part',
      'Type of Muscle',
      'Workout',
      'Burns_Calories_Bin']
```

1.3.4 Encodage ordinal

Les variables catégorielles ordonnées (par ex. Experience_Level, Difficulty Level) sont converties en valeurs entières selon leur progression logique.

Cette méthode conserve la hiérarchie implicite entre les modalités.

```
[72]: # Encodage ordinal sur variables à hiérarchie logique
ordinal_features = ["Experience_Level", "Difficulty Level"]

encoder_ordinal = OrdinalEncoder()
df_clean[ordinal_features] = encoder_ordinal.
    ↪ fit_transform(df_clean[ordinal_features])

print(" Encodage ordinal appliqué sur :", ordinal_features)
df_clean[ordinal_features].head()
```

Encodage ordinal appliqué sur : ['Experience_Level', 'Difficulty Level']

```
[72]: Experience_Level Difficulty Level
0          14.0          0.0
1          14.0          2.0
2           2.0          2.0
3          12.0          0.0
4          13.0          0.0
```

1.3.5 Encodage One-Hot

Les variables nominales (sans hiérarchie) sont transformées par One-Hot Encoding, créant une colonne binaire par modalité.

Cela permet d'éviter toute interprétation de relation d'ordre entre catégories.

```
[73]: df_encoded = pd.get_dummies(
    df_clean,
    columns=[col for col in cat_cols if col not in ["Experience_Level",
↪ "Difficulty Level"]],
    drop_first=True
)

print(f" One-Hot Encoding appliqué : {df_encoded.shape[1]} colonnes au total")
df_encoded.head(3)
```

One-Hot Encoding appliqué : 287 colonnes au total

```
[73]: Age Weight (kg) Height (m) Max_BPM Avg_BPM Resting_BPM \
0  34.91      65.27      1.62  188.58  157.65      69.05
1  23.37      56.41      1.55  179.43  131.75      73.18
2  33.20      58.98      1.67  175.04  123.95      54.96

Session_Duration (hours) Calories_Burned Fat_Percentage \
0              1.00      1080.90      26.800377
1              1.37      1809.91      27.655021
2              0.91       802.26      24.320821

Water_Intake (liters) ... Workout_Skull crushers \
0              1.50 ...              False
1              1.90 ...              False
2              1.88 ...              False

Workout_Standing calf raises Workout_Towel pull-up Workout_Triceps dips \
0              False              False              False
1              False              False              False
2              True              False              False

Workout_Triceps pushdowns Workout_Wrist curl Workout_Wrist extension \
0              False              False              False
1              False              False              False
```

	False	False	False
Burns_Calories_Bin_Low	Burns_Calories_Bin_Medium	\	
0	False	True	
1	False	False	
2	False	False	

Burns_Calories_Bin_Very High	
0	False
1	False
2	False

[3 rows x 287 columns]

1.3.6 Vérification post-encodage

Après encodage : - Toutes les variables sont numériques ; - Le nombre de colonnes augmente (selon le nombre de modalités) ; - Aucun NaN ne doit être réintroduit.

```
[74]: print(f"Forme finale du dataset encodé : {df_encoded.shape}")
      print(f"Valeurs manquantes totales : {df_encoded.isna().sum().sum()}")

      # Aperçu d'un sous-ensemble des nouvelles colonnes encodées
      df_encoded.filter(like="Workout_Type").head(5)
```

Forme finale du dataset encodé : (20000, 287)

Valeurs manquantes totales : 0

```
[74]: Workout_Type_HIIT  Workout_Type_Strength  Workout_Type_Yoga
0           False                True           False
1            True                False           False
2           False                False           False
3            True                False           False
4           False                True            False
```

1.3.7 Export du dataset encodé

On exporte le jeu de données final encodé pour les étapes suivantes : - Mise à l'échelle / normalisation - Split train/test

```
[75]: encoded_path = export_dir / "life_style_data_encoded.parquet"
      df_encoded.to_parquet(encoded_path, index=False)
      print(f" Export du dataset encodé : {encoded_path}")
```

Export du dataset encodé : c:\Users\fbax\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_encoded.parquet

L'encodage garantit la compatibilité du jeu de données avec les algorithmes de Machine Learning.

Les variables catégorielles hiérarchiques ont été traitées par encodage ordinal, et les variables nominales par encodage one-hot, réduisant les risques de biais d'interprétation.

1.4 Mise à l'échelle / Normalisation des variables numériques

1.4.1 Objectifs

Cette étape vise à homogénéiser les échelles des variables numériques afin d'éviter qu'une variable à grande amplitude (ex. Calories ou Weight (kg)) ne domine les autres lors de l'apprentissage du modèle. Deux techniques sont généralement utilisées : - StandardScaler : recentre les données autour de 0 et réduit leur variance à 1 ; - MinMaxScaler : ramène les valeurs dans un intervalle [0,1].

Le choix du scaler dépend de la nature des modèles : - Les algorithmes linéaires ou ceux utilisant une distance (SVM, KNN, PCA) sont sensibles à l'échelle → StandardScaler privilégié ; - Les modèles arborescents (Random Forest, XGBoost) y sont peu sensibles.

```
[76]: encoded_path = export_dir / "life_style_data_encoded.parquet"

df_encoded = pd.read_parquet(encoded_path)
print(f" Dataset encodé chargé : {df_encoded.shape[0]} lignes, {df_encoded.
      ↪shape[1]} colonnes")
df_encoded.head(3)
```

Dataset encodé chargé : 20000 lignes, 287 colonnes

```
[76]:
```

	Age	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	\
0	34.91	65.27	1.62	188.58	157.65	69.05	
1	23.37	56.41	1.55	179.43	131.75	73.18	
2	33.20	58.98	1.67	175.04	123.95	54.96	

	Session_Duration (hours)	Calories_Burned	Fat_Percentage	\
0	1.00	1080.90	26.800377	
1	1.37	1809.91	27.655021	
2	0.91	802.26	24.320821	

	Water_Intake (liters)	...	Workout_Skull crushers	\
0	1.50	...	False	
1	1.90	...	False	
2	1.88	...	False	

	Workout_Standing calf raises	Workout_Towel pull-up	Workout_Triceps dips	\
0	False	False	False	
1	False	False	False	
2	True	False	False	

	Workout_Triceps pushdowns	Workout_Wrist curl	Workout_Wrist extension	\
0	False	False	False	
1	False	False	False	

2	False	False	False
---	-------	-------	-------

	Burns_Calories_Bin_Low	Burns_Calories_Bin_Medium	\
0	False	True	
1	False	False	
2	False	False	

	Burns_Calories_Bin_Very High
0	False
1	False
2	False

[3 rows x 287 columns]

1.4.2 Sélection des variables numériques

On identifie toutes les variables numériques du dataset afin d'appliquer la mise à l'échelle uniquement sur celles qui ne sont pas déjà binaires (0/1).

Les colonnes issues du One-Hot Encoding ne doivent pas être modifiées.

```
[77]: # Identifier toutes les colonnes numériques
num_cols = df_encoded.select_dtypes(include=[np.number]).columns.tolist()

# Exclure les colonnes binaires (issues du One-Hot Encoding)
non_binary_num_cols = [col for col in num_cols if df_encoded[col].nunique() > 2]

print(f" Variables numériques à mettre à l'échelle : {len(non_binary_num_cols)}")
non_binary_num_cols[:15] # aperçu
```

Variables numériques à mettre à l'échelle : 40

```
[77]: ['Age',
'Weight (kg)',
'Height (m)',
'Max_BPM',
'Avg_BPM',
'Resting_BPM',
'Session_Duration (hours)',
'Calories_Burned',
'Fat_Percentage',
'Water_Intake (liters)',
'Workout_Frequency (days/week)',
'Experience_Level',
'BMI',
'Daily meals frequency',
'Physical exercise']
```

1.4.3 Application du StandardScaler

Le StandardScaler est appliqué sur toutes les variables continues identifiées.

Cette normalisation rend la moyenne nulle et la variance unitaire, garantissant un traitement équilibré par les algorithmes sensibles à l'échelle des features.

```
[78]: scaler = StandardScaler()
scaled_values = scaler.fit_transform(df_encoded[non_binary_num_cols])

# Remplacer les colonnes originales par leurs versions normalisées
df_scaled = df_encoded.copy()
df_scaled[non_binary_num_cols] = scaled_values

print(" Standardisation terminée.")
df_scaled[non_binary_num_cols].describe().T.head(10)
```

Standardisation terminée.

```
[78]:
```

	count	mean	std	min	25%	\
Age	20000.0	-4.261480e-16	1.000025	-1.721230	-0.881724	
Weight (kg)	20000.0	7.105427e-19	1.000025	-1.639809	-0.743363	
Height (m)	20000.0	-4.213518e-16	1.000025	-1.834954	-0.811571	
Max_BPM	20000.0	1.811884e-17	1.000025	-1.787904	-0.854193	
Avg_BPM	20000.0	-1.726619e-16	1.000025	-1.726623	-0.875027	
Resting_BPM	20000.0	-1.524114e-16	1.000025	-1.743109	-0.855490	
Session_Duration (hours)	20000.0	2.572165e-16	1.000025	-2.254275	-0.613623	
Calories_Burned	20000.0	-3.403500e-16	1.000025	-1.905552	-0.735359	
Fat_Percentage	20000.0	-3.019807e-16	1.000025	-2.955980	-0.743291	
Water_Intake (liters)	20000.0	-2.280842e-16	1.000025	-1.930657	-0.756538	

	50%	75%	max
Age	0.083665	0.889739	1.718513
Weight (kg)	-0.184146	0.576275	2.686089
Height (m)	-0.103074	0.605422	2.258579
Max_BPM	0.021745	0.828399	1.715848
Avg_BPM	-0.050066	0.866013	1.831856
Resting_BPM	0.000574	0.808623	1.688010
Session_Duration (hours)	0.030919	0.587569	2.228221
Calories_Burned	-0.096890	0.543596	3.207204
Fat_Percentage	-0.055806	0.715510	1.781150
Water_Intake (liters)	-0.028916	0.814466	1.823215

1.4.4 Vérification post-normalisation

Après mise à l'échelle : - La moyenne des colonnes est proche de 0 ; - L'écart-type est proche de 1 ; - Les colonnes binaires n'ont pas été modifiées.


```
[79]: # Vérification moyenne ~0 et std ~1
check_stats = df_scaled[non_binary_num_cols].agg(["mean", "std"]).T
check_stats.head(10)
```

```
[79]:
```

	mean	std
Age	-4.261480e-16	1.000025
Weight (kg)	7.105427e-19	1.000025
Height (m)	-4.213518e-16	1.000025
Max_BPM	1.811884e-17	1.000025
Avg_BPM	-1.726619e-16	1.000025
Resting_BPM	-1.524114e-16	1.000025
Session_Duration (hours)	2.572165e-16	1.000025
Calories_Burned	-3.403500e-16	1.000025
Fat_Percentage	-3.019807e-16	1.000025
Water_Intake (liters)	-2.280842e-16	1.000025

1.4.5 Export du dataset normalisé

Le dataset final normalisé est sauvegardé pour la suite du pipeline :
 - life_style_data_scaled.parquet : pour la modélisation supervisée ;
 - life_style_data_scaled.csv : pour inspection rapide sous DBEaver.

```
[80]: scaled_path = export_dir / "life_style_data_scaled.parquet"
csv_scaled_path = export_dir / "life_style_data_scaled.csv"

df_scaled.to_parquet(scaled_path, index=False)
df_scaled.to_csv(csv_scaled_path, index=False)

print(f" Export parquet : {scaled_path}")
print(f" Export csv      : {csv_scaled_path}")
print(f" Shape final      : {df_scaled.shape}")
```

Export parquet : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_scaled.parquet

Export csv : c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\data\processed\life_style_data\life_style_data_scaled.csv

Shape final : (20000, 287)

Critères de validation : - Moyenne 0 et écart-type 1 pour les colonnes normalisées ; - Aucune valeur manquante ou anormale après transformation ; - Colonnes binaires conservées inchangées ; - Fichiers d'export générés sans perte de structure.

Cette étape garantit une échelle cohérente entre toutes les variables continues, assurant un apprentissage plus stable et une meilleure convergence lors de la modélisation.

2 Code complet avec pipeline

```
[81]: # pipeline = Pipeline(steps=[  
#     ('imputer', SimpleImputer(strategy='median')),  
#     ('scaler', StandardScaler()),  
#     ('model', LinearRegression())  
# ])
```

```
[82]: # pipeline.fit(X_train, y_train)
```

```
[83]: # y_pred = pipeline.predict(X_test)  
# pd.DataFrame(y_pred, columns=['Predicted_PRICE'])
```